

Reviewer Pack — Minimal Index of Diophantine Equivalence and Resolution Proo...

Entry 1ecdbd4e7f62 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 18:40:18 UTC

Minimal Index of Diophantine Equivalence and Resolution Proof Depth

Entry ID: 1ecdbd4e7f62 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 18:40:18 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equivalence) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

For every Tseitin formula φ with n variables, the minimal index of diophantine equivalence ($ID(\varphi)$) of its associated Diophantine set is linearly correlated with its resolution proof depth $d(\varphi)$, such that $ID(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Diophantine equations have been used to represent combinatorial problems, and their solvability can be related to the complexity of these problems. The minimal index of diophantine equivalence quantifies the difficulty of solving a Diophantine equation and may provide insights into the resolution proof depth.

Taxonomy category: Diophantine_Equivalence_Resolution (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 14c20a9e17eb03d1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the ratios of minimal index of diophantine equivalence ($ID(\varphi)$) to resolution proof depth ($d(\varphi)$) across 30 random seeds is within a range $[0.5, 1.5]$, and no seed produces a ratio outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def mod_inverse(a, m):
        for x in range(1, m):
            if (a * x) % m == 1:
                return x
        raise ValueError("Modular inverse does not exist")

    def diophantine_set(clauses):
        n = len(clauses)
        equations = []
        mod = 2**n + 1 # A prime number greater than n

        for clause in clauses:
            equation = 0
            for literal, sign in clause:
                index = abs(literal) - 1
                coefficient = sign * (-1 if literal < 0 else 1)
                try:
                    equation += coefficient * mod_inverse(index + 1, mod)
                except ValueError:
                    return None # Return None if modular inverse does not exist
            equations.append(equation % mod)

        return equations

    def resolution_proof_depth(clauses):
        n = len(clauses)
        stack = clauses[:]
        depth = 0

        while stack:
```

```

        clause1 = stack.pop()
        for clause2 in stack:
            new_clause = []
            for literal1 in clause1:
                if -literal1 in clause2:
                    common_literals = [l for l in clause1 if l != literal1]
                    new_clause.extend(common_literals)
                    break
            else:
                continue
            new_clause = list(set(new_clause))
            if not new_clause:
                return depth + 1
            stack.append(new_clause)
        depth += 1

    return depth

def tseitin_formula(n):
    variables = [f'x{i+1}' for i in range(n)]
    clauses = []

    # Clause for each variable being true or false
    for var in variables:
        clause = [(var, 1), (-var, -1)]
        clauses.append(clause)

    # Clause for each pair of variables being different
    for i in range(n):
        for j in range(i + 1, n):
            clause = [(-variables[i], 1), (variables[j], 1), (-variables[j], -1), (variables[i], -1)]
            clauses.append(clause)

    # Clause for each pair of variables being the same
    for i in range(n):
        for j in range(i + 1, n):
            clause = [(variables[i], 1), (variables[j], -1), (-variables[j], 1), (-variables[i], -1)]
            clauses.append(clause)

    return clauses

n = random.randint(5, 40)
clauses = tseitin_formula(n)

diophantine_eqs = diophantine_set(clauses)
if diophantine_eqs is None:
    return {
        "metric_name": "ID( $\phi$ )/d( $\phi$ )",
        "metric_value": float('nan'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "modular_inverse_does_not_exist"
    }

proof_depth = resolution_proof_depth(clauses)

```

```

if proof_depth == 0:
    return {
        "metric_name": "ID( $\phi$ )/d( $\phi$ )",
        "metric_value": float('nan'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "resolution_proof_depth_zero"
    }

ratio = Fraction(diophantine_eqs[0], proof_depth)

return {
    "metric_name": "ID( $\phi$ )/d( $\phi$ )",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results if not math.isnan(result["metric_value"])) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(not math.isnan(result["metric_value"]) for result in results) and 0.5 <= mean_ratio <= 1.0:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=nan support_fraction={support_fraction}")
    elif any(math.isnan(result["metric_value"]) for result in results):
        print("RESULT: INCONCLUSIVE metric_saturation")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='ratio_outside_range' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79db13c3.py", line 134, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79db13c3.py", line 93, in run_trial
    clauses = tseitin_formula(n)

```

```

^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_79db13c3.py", line 75, in tseitin_formu
    clause = [(var, 1), (-var, -1)]
              ^^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended task of verifying the conjecture. | next: Investigate and fix the error in the test code to ensure it can run to completion and provide the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12640
2	preregistration	ollama_remote	glm4:latest	0	0	9576
3	novelty	ollama_remote	glm4:latest	0	0	15522
4	novelty	ollama_remote	glm4:latest	0	0	9178
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18761
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14364
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13440
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14509
9	judge	ollama_remote	glm4:latest	0	0	21164

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129154 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1ecdbd4e7f62.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1ecdbd4e7f62.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1ecdbd4e7f62.tar.gz` (if generated)